

Assignment 1:

Course: Machine Learning

Student: Navneet Das

Python Refresher for Advanced Machine Learning

Question 1: Python Fundamentals and Decision Making

In [40]: *# Declare variables of different types (int, float, str, bool) and print their types using type()*

```
variable_1 = 10
variable_2 = 10.0
variable_3 = "Ten"
variable_4 = True

print(f"variable_1 :{type(variable_1)}")
print(f"variable_2 :{type(variable_2)}")
print(f"variable_3 :{type(variable_3)}")
print(f"variable_4 :{type(variable_4)}")
```

```
variable_1 :<class 'int'>
variable_2 :<class 'float'>
variable_3 :<class 'str'>
variable_4 :<class 'bool'>
```

In [47]: *# Take a student's marks as input and classify the grade using if-elif-else (e.g., A/B/C/F).*

```
marks = float(input("Enter your marks:"))
if marks >= 90.0:
    print("GRADE: A")
elif marks >= 80.0:
    print("GRADE: B")
elif marks >= 70.0:
    print("GRADE: C")
else:
    print("GRADE: F")
```

GRADE: F

In [48]: *# Evaluate a few arithmetic expressions and observe operator precedence*

```
# Expression 1: Mixed Operators (*, /, +)
expr1 = 10 + 4 * 5 / 2
print(expr1)
```

20.0

In [49]: *# Expression 2: Overriding Precedence with Parentheses*

```
expr2 = (10 + 4) * 5 / 2
print(expr2)
```

35.0

In [50]: *expr3 = 2 ** 3 ** 2*

```
print(expr3)
```

512

Priority

Operator Level	Syntax	Associativity
Highest	() (Parentheses)	Left-to-Right
High	** (Exponentiation)	Right-to-Left
Medium	*, /, //, % (Multiplication & Divisions)	Left-to-Right
Lowest	+, - (Addition & Subtraction)	Left-to-Right

Question 2: Loops and Functions

```
In [51]: # Print numbers 1 to N using a for loop, then print only the even numbers.
a = [i for i in range(20) if i%2 == 0]
print(a)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

```
In [53]: # Compute the factorial of a number using a loop.
num = int(input("Enter The Number you Want a Factor of :"))
factors = [i for i in range(1,num) if num%i == 0]
factors
```

```
Out[53]: [1, 2, 4, 8, 16, 32]
```

```
In [54]: # Define functions square(x) and cube(x).
def square(num):
    return num * num
def cube(num):
    return num**3

print(square(42))
print(cube(42))
```

```
1764
74088
```

```
In [55]: # Define euclidean_distance(point1, point2) to compute the distance between two n-dimensional
import math
def euclidean_distance(x1, x2, y1, y2):
    return math.sqrt((x2-x1)**2+(y2-y1)**2)

print(euclidean_distance(0,6,4,2))
```

```
6.324555320336759
```

```
In [56]: # Define normalize(values) that scales a list of numbers to the range [0, 1].
def normalize(mylist):
    norm = [i/max(mylist) for i in mylist]
    return norm

li=[11,54,67,84,35,26,42,97,62,15,9]
print(normalize(li))
```

```
[0.1134020618556701, 0.5567010309278351, 0.6907216494845361, 0.865979381443299, 0.360824742268
04123, 0.26804123711340205, 0.4329896907216495, 1.0, 0.6391752577319587, 0.15463917525773196,
0.09278350515463918]
```

Question 3: Data Structures

```
In [57]: # Create a list of student names and marks, and sort it by marks
students = [
    ("navneet", 85),
    ("chandra", 98),
    ("vicky", 58),
    ("akshat", 78),
    ("ayush", 94),
    ("manish", 65),
    ("vivek", 89),
    ("tripti", 72)
]

sorted_students = sorted(students, key=lambda x: x[1], reverse=True)

# 3. Print the sorted results
for rank, (name, mark) in enumerate(sorted_students, 1):
    print(f"{rank}. {name}: {mark}")
```

```
1. chandra: 98
2. ayush: 94
3. vivek: 89
4. navneet: 85
5. akshat: 78
6. tripti: 72
7. manish: 65
8. vicky: 58
```

```
In [58]: # Create a dictionary mapping student name to marks; retrieve and update values
student_marks = {
    "navneet": 85,
    "akshat": 92,
    "vicky": 78
}

print("Initial Dictionary:", student_marks)

navneet_marks = student_marks["navneet"]
print(f"navneet Marks: {navneet_marks}")

clara_marks = student_marks.get("Clara", "Not Found")
print(f"Clara's Marks: {clara_marks}")

student_marks["akshat"] = 95
print("After updating akshat's marks:", student_marks)

student_marks["vicky"] = 88
print("After adding vicky:", student_marks)
```

```
Initial Dictionary: {'navneet': 85, 'akshat': 92, 'vicky': 78}
navneet Marks: 85
Clara's Marks: Not Found
After updating akshat's marks: {'navneet': 85, 'akshat': 95, 'vicky': 78}
After adding vicky: {'navneet': 85, 'akshat': 95, 'vicky': 88}
```

```
In [59]: # Create a set from a list of feature names containing duplicates to demonstrate de-duplication
feature_names = ["age", "income", "age", "education", "income", "credit_score"]
unique_features = set(feature_names)
print(f"Original list (with duplicates): {feature_names}")
print(f"Resulting set (de-duplicated): {unique_features}")
```

Original list (with duplicates): ['age', 'income', 'age', 'education', 'income', 'credit_score']

Resulting set (de-duplicated): {'education', 'income', 'age', 'credit_score'}

```
In [60]: # Use a list comprehension to derive a list of letter grades from the marks list.
marks = [92, 78, 85, 45, 60, 33]
def get_grade(m):
    if m >= 90: return 'A'
    elif m >= 80: return 'B'
    elif m >= 70: return 'C'
    elif m >= 60: return 'D'
    else: return 'F'
grades = [get_grade(mark) for mark in marks]
print(grades)
```

['A', 'C', 'B', 'F', 'D', 'F']

Question 4: Numerical Computing with NumPy

```
In [61]: import numpy as np
```

```
In [62]: # Create 1D and 2D NumPy arrays; inspect their shape, dtype and size
array_1d = np.array([1,2,3,4,5,6,7,8,9])
array_2d = np.array([array_1d,array_1d])
print(f"1d array = {array_1d} \nshape:{array_1d.shape}\nsize:{array_1d.size}")
print(f"\n{"-"*40}\n")
print(f"2d array = {array_2d} \nshape:{array_2d.shape}\nsize:{array_2d.size}")
```

1d array = [1 2 3 4 5 6 7 8 9]
shape:(9,)
size:9

2d array = [[1 2 3 4 5 6 7 8 9]
[1 2 3 4 5 6 7 8 9]]
shape:(2, 9)
size:18

```
In [63]: # Compute mean, standard deviation, minimum, maximum and sum, including along specific axes
print(f" maximum: {array_2d.max()}")
print(f" minimum: {array_2d.min()}")
print(f" sum: {array_2d.sum()}")
print(f" mean: {array_2d.mean()}")
print(f" Standard deviation: {array_2d.std()}")

print(f"\nanother way{"-"*40}\n")
# another way
print("Sum:", np.sum(array_2d))           # Total sum: 21
print("Mean:", np.mean(array_2d))         # Average: 3.5
print("Std Dev:", np.std(array_2d))        # Standard deviation: 1.7078
print("Minimum:", np.min(array_2d))        # Smallest value: 1
print("Maximum:", np.max(array_2d))        # Largest value: 6

print(f"\nwith axis{"-"*40}\n")
# Sum along columns (axis=0) and rows (axis=1)
print("Sum down columns (axis=0):", np.sum(array_2d, axis=0))
print("Sum across rows (axis=1):", np.sum(array_2d, axis=1))

# Mean, Min, Max, and Std along axes
print("Mean along axis=0:", np.mean(array_2d, axis=0))
print("Max along axis=1:", np.max(array_2d, axis=1))
```

```
print("Min along axis=0:", np.min(array_2d, axis=0))
print("Std along axis=1:", np.std(array_2d, axis=1))
```

```
maximum: 9
minimum: 1
sum: 90
mean: 5.0
Standard deviation: 2.581988897471611
```

another way-----

```
Sum: 90
Mean: 5.0
Std Dev: 2.581988897471611
Minimum: 1
Maximum: 9
```

with axis-----

```
Sum down columns (axis=0): [ 2  4  6  8 10 12 14 16 18]
Sum across rows (axis=1): [45 45]
Mean along axis=0: [1. 2. 3. 4. 5. 6. 7. 8. 9.]
Max along axis=1: [9 9]
Min along axis=0: [1 2 3 4 5 6 7 8 9]
Std along axis=1: [2.5819889 2.5819889]
```

In [64]: *# Perform matrix operations: transpose and matrix multiplication (dot product)*

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([[7, 8],
              [9, 10],
              [11, 12]])

# 2. Matrix Transpose
A_transposed = A.T
print("Transpose of A:\n", A_transposed)

# 3. Matrix Multiplication (Dot Product) @ operator
result_operator = A @ B
print("\nMatrix Multiplication (A @ B):\n", result_operator)

# Alternative standard syntax: np.dot()
result_dot = np.dot(A, B)
print(result_dot)
```

Transpose of A:

```
[[1 4]
 [2 5]
 [3 6]]
```

Matrix Multiplication (A @ B):

```
[[ 58  64]
 [139 154]]
[[ 58  64]
 [139 154]]
```

In [65]: *# Generate random numbers using numpy.random, and compare the time taken for a vectorized open*

```
import time
n = 1_000_000

# 1. Python Loop Approach
start_time = time.time()
python_list = []
```

```

for _ in range(n):
    python_list.append(np.random.rand())
loop_time = time.time() - start_time

# 2. Vectorized NumPy Approach
start_time = time.time()
numpy_arr = np.random.rand(n)
vector_time = time.time() - start_time

print(f"Python Loop Time: {loop_time:.4f} seconds")
print(f"Vectorized NumPy Time: {vector_time:.4f} seconds")
print(f"Speedup: {loop_time / vector_time:.1f}x faster")

```

Python Loop Time: 0.5660 seconds
 Vectorized NumPy Time: 0.0067 seconds
 Speedup: 84.8x faster

Question 5: Data Manipulation and Visualization with Pandas

```

In [66]: # Create a DataFrame with columns Name, Marks, Height and Weight
import pandas as pd
data = {
    'Name': ['navneet', 'prince', 'hemant', "vicky"],
    'Marks': [85, 90, 48, 50],
    'Height': [165, 175, 160, 140],
    'Weight': [55, 70, 62, 51]
}
df = pd.DataFrame(data)
df

```

```

Out[66]:

```

	Name	Marks	Height	Weight
0	navneet	85	165	55
1	prince	90	175	70
2	hemant	48	160	62
3	vicky	50	140	51

```

In [67]: # Filter students who scored above a chosen threshold, and add a derived Result column (Pass/Fail)
threshold = 60

# Syntax: np.where(condition, value_if_true, value_if_false)
df["Result"] = np.where(df["Marks"] >= threshold, "Pass", "Fail")

passed_students = df[df["Result"] == "Pass"]
print("--- Original DataFrame with Result Column ---")
print(df)
print("\n--- Filtered DataFrame (Passed Students Only) ---")
print(passed_students)

```

--- Original DataFrame with Result Column ---

	Name	Marks	Height	Weight	Result
0	navneet	85	165	55	Pass
1	prince	90	175	70	Pass
2	hemant	48	160	62	Fail
3	vicky	50	140	51	Fail

--- Filtered DataFrame (Passed Students Only) ---

	Name	Marks	Height	Weight	Result
0	navneet	85	165	55	Pass
1	prince	90	175	70	Pass

```
In [68]: # Group the data by Result and compute summary statistics for each group.
z = df.groupby("Result")
z.get_group("Pass").describe()
```

```
Out[68]:
```

	Marks	Height	Weight
count	2.000000	2.000000	2.000000
mean	87.500000	170.000000	62.500000
std	3.535534	7.071068	10.606602
min	85.000000	165.000000	55.000000
25%	86.250000	167.500000	58.750000
50%	87.500000	170.000000	62.500000
75%	88.750000	172.500000	66.250000
max	90.000000	175.000000	70.000000

```
In [69]: # Group the data by Result and compute summary statistics for each group.
z = df.groupby("Result")
z.get_group("Fail").describe()
```

```
Out[69]:
```

	Marks	Height	Weight
count	2.000000	2.000000	2.000000
mean	49.000000	150.000000	56.500000
std	1.414214	14.142136	7.778175
min	48.000000	140.000000	51.000000
25%	48.500000	145.000000	53.750000
50%	49.000000	150.000000	56.500000
75%	49.500000	155.000000	59.250000
max	50.000000	160.000000	62.000000

```
In [70]: # Create a Line plot, a bar plot and a histogram to visualize different aspects of the data.
import matplotlib.pyplot as plt

# Set up a 1x3 grid for side-by-side plots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# -----
# 1. Line Plot: Height and Weight Trends across Students
# -----
```

```

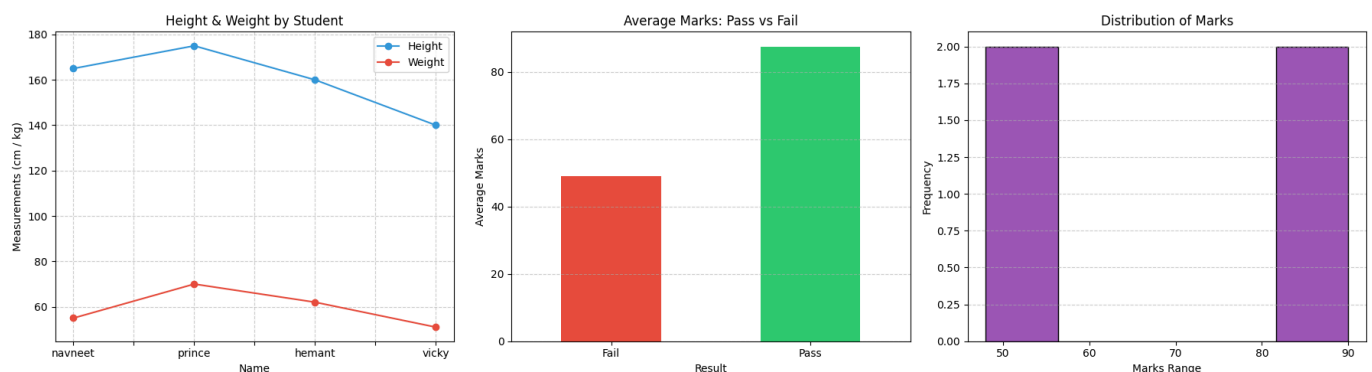
df.plot(
    kind="line",
    x="Name",
    y=["Height", "Weight"],
    marker="o",
    ax=axes[0],
    color=["#3498db", "#e74c3c"],
)
axes[0].set_title("Height & Weight by Student")
axes[0].set_ylabel("Measurements (cm / kg)")
axes[0].grid(True, linestyle="--", alpha=0.6)

# -----
# 2. Bar Plot: Average Marks by Result Group
# -----
avg_marks = df.groupby("Result")["Marks"].mean()
avg_marks.plot(kind="bar", ax=axes[1], color=["#e74c3c", "#2ecc71"])
axes[1].set_title("Average Marks: Pass vs Fail")
axes[1].set_ylabel("Average Marks")
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=0)
axes[1].grid(axis="y", linestyle="--", alpha=0.6)

# -----
# 3. Histogram: Distribution of Student Marks
# -----
df["Marks"].plot(
    kind="hist", bins=5, ax=axes[2], color="#9b59b6", edgecolor="black"
)
axes[2].set_title("Distribution of Marks")
axes[2].set_xlabel("Marks Range")
axes[2].grid(axis="y", linestyle="--", alpha=0.6)

# Adjust layout and display charts
plt.tight_layout()
plt.show()

```



Question 6: Mini ML Exercise — Preparing Data for Machine Learning

In [73]: `# Normalize the Height and Weight columns using min-max scaling (reuse your normalize() function)`

```

print(normalize(df["Height"]))
normalize(df["Weight"])

```

[0.9428571428571428, 1.0, 0.9142857142857143, 0.8]

Out[73]: [0.7857142857142857, 1.0, 0.8857142857142857, 0.7285714285714285]

In [82]: `# Compute the correlation between Height and Weight (and, optionally, Marks).`

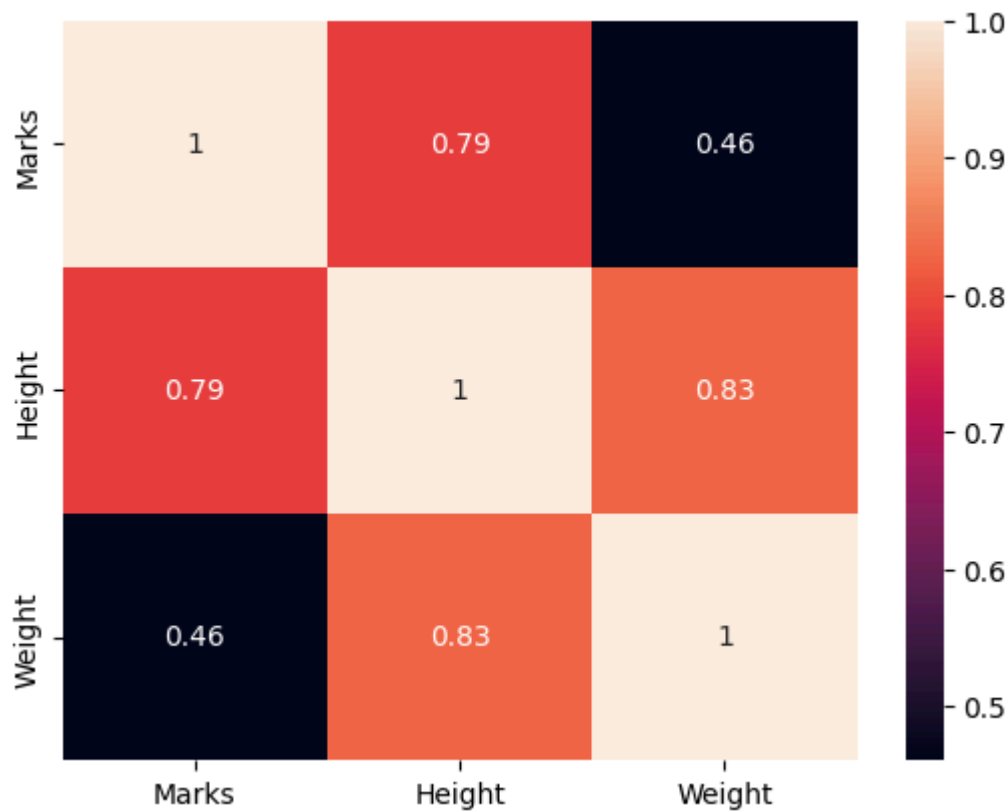
```

import seaborn as sns

```

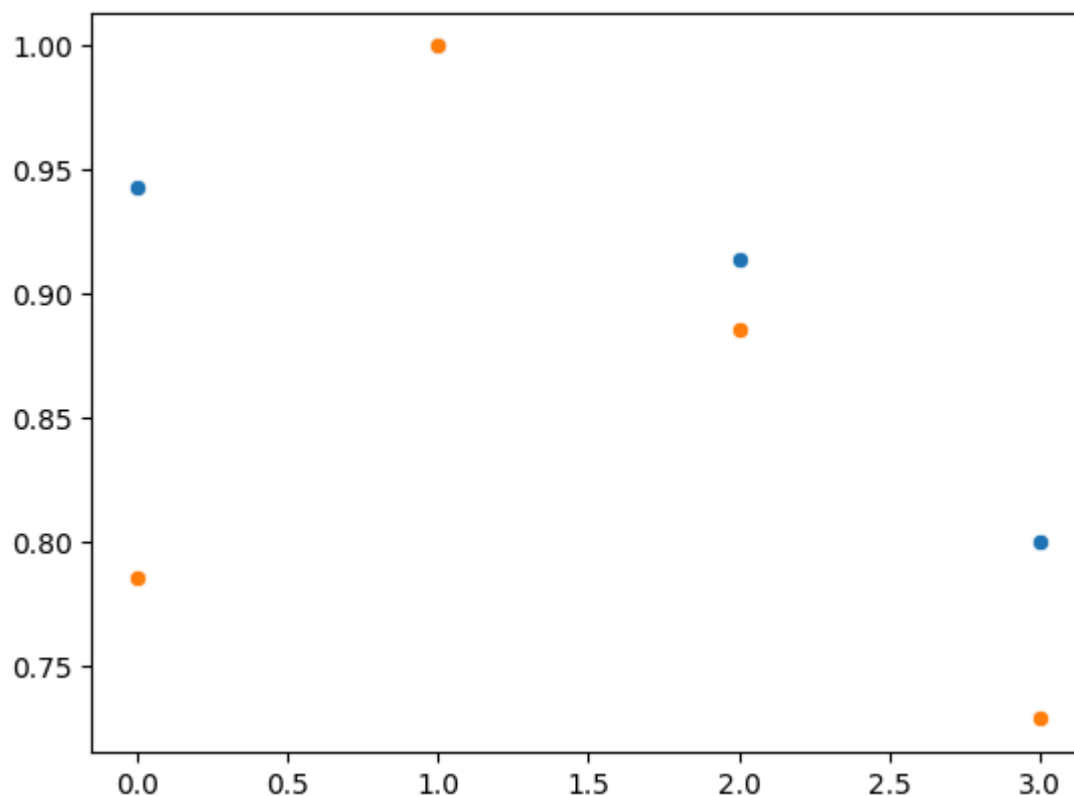
```
sns.heatmap(df.iloc[:,1:-1].corr(),annot=True)
```

Out[82]: <Axes: >



In [85]: *# Create a scatter plot of normalized Height vs. Weight and comment on any visible relationships*
`sns.scatterplot(normalize(df["Height"]))`
`sns.scatterplot(normalize(df["Weight"]))`

Out[85]: <Axes: >



Discuss why normalization is generally necessary before using distance-based or gradient-based ML algorithms

Algorithms like **K-Nearest Neighbors (KNN)**, **K-Means clustering**, and **Support Vector Machines (SVM)** calculate the distance (such as Euclidean distance) between data points.

Preventing dominance: If one feature has a large numerical range (like annual salary in dollars from \$30,000 to \$150,000) and another has a small range (like age from 20 to 80), the larger numbers will completely control the distance calculation.

Ensuring fair contribution: Normalizing forces all features into the same range (typically 0 to 1), allowing every feature to contribute equally to the final similarity or distance score